



UNIVERZITET U NIŠU
ELEKTRONSKI FAKULTET



Buffer pool i caching mehanizmi

Seminarski rad
Studijski program: Elektrotehnika i računarstvo
Modul: Bezbednost računarskih sistema
Predmet: Sistemi za upravljanje bazama podataka

Student:

Katarina Adamović, br. ind. 2173

Mentor:

Prof. Aleksandar Stanimirović

Niš, maj 2026.

SADRŽAJ:

1. Uvod	4
2. Storage engine i fizička organizacija podataka.....	5
2.1 Uloga storage engine-a u arhitekturi DBMS-a	5
2.2 Fizička organizacija podataka.....	5
2.2.1 Heap storage model	6
2.2.2 Stranice (Pages)	6
2.3 Disk i memorija – razlika u brzini	6
2.4 Disk I/O operacije.....	7
2.5 Sekvencijalni i nasumični pristup podacima	7
2.6 Read/Write putanja podataka.....	7
3. Buffer pool.....	8
3.1 Osnovni koncepti buffer pool-a	8
3.1.1 Keširanje stranica.....	8
3.1.2 Smanjenje I/O operacija	9
3.1.3 Dirty pages.....	9
3.2 Cache hit i cache miss.....	9
3.3 Upravljanje buffer pool-om	9
3.3.1 Politike zamene stranica (evikcija).....	9
3.3.2 LRU, MRU i CLOCK algoritmi	10
3.3.3 Free list i midpoint insertion.....	10
3.3.4 OS page cache sloj.....	11
3.4 Arhitektura buffer pool-a.....	11
3.4.1 Interna organizacija – page table i frame metapodaci	11
3.4.2 Buffer Pool Extension (BPE).....	12
3.4.3 Konkurentnost i latch mehanizmi	12
3.4.4 Uticaj na performanse.....	12
3.5 Životni ciklus stranice u buffer pool-u	13
4. Caching mehanizam i uticaj na performanse.....	14
4.1 Slojevitost memorijske hijerarhije.....	14
4.2 Cold cache i warm cache	15
4.3 Caching mehanizmi pored buffer pool-a	15
4.3.1 Plan cache (keš planova izvršavanja)	15
4.3.2 OS page cache i interakcija sa shared_buffers.....	15
4.3.3 pg_preload_wal – predgreivanje buffer pool-a	16
4.4 Efekat ponovljenih upita i buffer hit ratio	16
4.5 Povezanost sa performansama sistema.....	16
4.5.1 OLTP vs OLAP workload	16
4.5.2 Keširanje i optimizator upita	17
4.5.3 Caching i konkurentni pristup	17

5. Prakticni primeri	17
5.1 Testiranje manje baze	17
5.1.1 Okruženje i konfiguracija	17
5.1.2 Sekvencijalno skeniranje bez indeksa (Seq Scan)	18
5.1.3 . Indeksno skeniranje — cold cache i warm cache	19
5.1.3.1 Prvo izvršavanje — cold cache.....	20
5.1.3.2 Drugo izvršavanje — warm cache.....	20
5.1.4 Poređenje: Seq Scan vs Index Scan (cold vs warm).....	21
5.1.5 Uticaj selektivnosti na izbor plana izvršavanja.....	21
5.2 Testiranje veće baze.....	23
5.2.1 Okruženje i konfiguracija	23
5.2.2 Seq Scan bez indeksa.....	23
5.2.3 Kreiranje indeksa, pa opet:	24
5.2.3.1 Prvo izvršavanje — cold cache.....	24
5.2.3.2 Drugo izvršavanje — warm cache.....	26
5.2.4 Uticaj selektivnosti na izbor plana izvršavanja.....	26
5.3 Uticaj veličine tabele u odnosu na shared_buffers	28
6. Zaključak	29
7. Literatura.....	30

1. Uvod

Sistemi za upravljanje bazama podataka predstavljaju osnovu velikog broja savremenih informacionih sistema. Sa rastom količine podataka i broja korisničkih zahteva, performanse baze podataka postaju jedan od najvažnijih faktora u radu aplikacija. Brzina izvršavanja upita u velikoj meri zavisi od načina na koji su podaci organizovani na disku, kao i od načina na koji baza upravlja memorijom tokom rada.

Jedan od osnovnih problema sa kojima se sistemi baza podataka suočavaju jeste razlika u brzini između glavne memorije i sekundarne memorije. Iako procesor i RAM memorija omogućavaju veoma brz pristup podacima, trajno skladištenje podataka vrši se na disku, čiji je pristup višestruko sporiji. Zbog toga veliki broj operacija nad bazom podataka zavisi od optimizacije pristupa podacima i smanjenja broja disk operacija, odnosno I/O operacija.

U relacionim bazama podataka kao što je PostgreSQL, podaci se fizički organizuju kroz stranice (pages) i skladište na disku u okviru posebnih data fajlova. Sistem zatim koristi različite mehanizme kako bi omogućio što efikasnije čitanje i upis podataka. Među najvažnijim komponentama zaduženim za ove operacije nalaze se storage engine i caching mehanizmi.

Storage engine predstavlja sloj sistema baze podataka koji upravlja fizičkom organizacijom podataka, pristupom fajlovima na disku, načinom čuvanja stranica podataka i komunikacijom između memorije i skladišta podataka. Njegova uloga je da obezbedi pouzdan i efikasan pristup podacima uz minimalne troškove izvršavanja operacija. Pored storage engine-a, važnu ulogu ima i buffer pool, odnosno memorijski prostor rezervisan za keširanje često korišćenih stranica podataka. Korišćenjem buffer pool-a sistem može izbeći veliki broj sporih pristupa disku i značajno ubrzati izvršavanje SQL upita.

Pored samog keširanja podataka, sistemi baza podataka koriste i različite metode optimizacije pristupa podacima, kao što su indeksi, organizacija podataka po stranicama, kao i optimizacija planova izvršavanja upita. Način na koji baza podataka bira strategiju izvršavanja upita direktno utiče na performanse sistema, posebno kod rada sa velikim količinama podataka.

Cilj ovog rada jeste analiza storage engine mehanizama i buffer pool sistema u PostgreSQL bazi podataka, kao i ispitivanje njihovog uticaja na performanse izvršavanja upita. Poseban fokus biće na analizi pristupa podacima sa diska i iz memorije, kao i na načinu na koji keširanje utiče na smanjenje I/O operacija i ubrzavanje rada sistema.

U praktičnom delu rada biće korišćeni SQL upiti i EXPLAIN ANALYZE mehanizam radi analize različitih planova izvršavanja upita. Biće prikazano ponašanje sistema pri sequential scan, bitmap heap scan i index scan operacijama, kao i razlike u vremenu izvršavanja upita pri radu sa podacima koji se nalaze u memoriji i podacima koji se učitavaju sa diska. Na osnovu dobijenih rezultata biće izvršena analiza efikasnosti buffer pool i caching mehanizama u PostgreSQL sistemu.

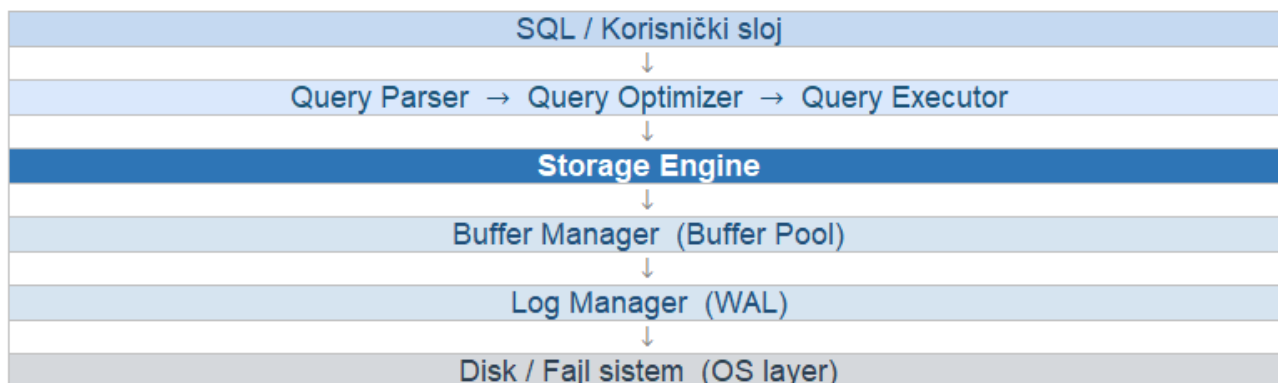
2. Storage engine i fizička organizacija podataka

Storage engine predstavlja najniži logički sloj arhitekture sistema za upravljanje bazama podataka, smešten između operativnog sistema i viših komponenti kao što su query optimizer i executor. Njegova osnovna uloga je upravljanje fizičkom organizacijom podataka – načinom na koji se podaci čuvaju na disku i kako se efikasno dohvataju tokom izvršavanja operacija.

Za razliku od viših slojeva koji rade sa logičkim strukturama kao što su tabele i relacije, storage engine direktno upravlja fajlovima, stranicama podataka i I/O operacijama. Pošto je pristup podacima sa diska višestruko sporiji u odnosu na RAM memoriju, efikasnost storage engine-a ima direktan uticaj na ukupne performanse sistema.

2.1 Uloga storage engine-a u arhitekturi DBMS-a

Storage engine je komponenta DBMS-a koja izvršava sve operacije fizičkog pristupa podacima. U okviru PostgreSQL-a, ovaj sloj ne obrađuje SQL upite direktno – umesto toga, prima instrukcije od query executora u obliku planova izvršavanja i nad njima obavlja konkretne operacije čitanja i pisanja.



Svaki sloj arhitekture ima jasno definisanu ulogu. Query optimizer generiše plan izvršavanja; executor prolazi kroz taj plan; storage engine fizički čita ili upisuje podatke; buffer manager odlučuje da li se podaci nalaze u RAM-u ili treba da se učitaju sa diska.

Zašto PostgreSQL nema plugable storage engine? Za razliku od MySQL-a koji podržava zamenjive storage engine-e (InnoDB, MyISAM, RocksDB), PostgreSQL koristi jedinstven, čvrsto integrisan storage sloj zasnovan na heap storage modelu. Ovo omogućava dublju optimizaciju između slojeva, ali smanjuje fleksibilnost izbora. Od verzije 12, PostgreSQL uvodi Table Access Method API koji otvara vrata plugable storage-u u budućnosti.

2.2 Fizička organizacija podataka

Fizička organizacija podataka opisuje način na koji su podaci stvarno smešteni na disku, nezavisno od logičke strukture koju korisnik vidi kroz tabele i relacije. Dok se u SQL jeziku podaci prikazuju kao tabele sa kolonama i redovima, na fizičkom nivou oni se organizuju kroz binarne strukture optimizovane za efikasno čitanje i pisanje.

2.2.1 Heap storage model

U PostgreSQL-u, podrazumevani model fizičke organizacije je heap storage. U ovom modelu, redovi podataka (tuples) smeštaju se u prvi dostupan slobodan prostor, bez održavanja sortiranog redosleda. Ovakav pristup omogućava jednostavno i brzo umetanje novih podataka, ali zahteva dodatne mehanizme (indekse) za efikasno pretraživanje.

Svaka tabela u PostgreSQL-u se čuva u jednom ili više fizičkih fajlova na disku. Ovi fajlovi nisu obični tekstualni zapisi, već binarne strukture organizovane u stranice fiksne veličine. Sistem fajlova operativnog sistema obezbeđuje osnovni pristup disku, dok PostgreSQL upravlja organizacijom podataka unutar tih fajlova.

2.2.2 Stranice (Pages)

Stranica (page) je osnovna jedinica organizacije i prenosa podataka između diska i memorije. PostgreSQL koristi stranice veličine 8 KB – svaki pristup disku uvek učitava ili upisuje celu stranicu, nikad manji deo.

Fajl na disku	Stranice (Pages)	Sadržaj stranice
customers.dat	Page 0 (8 KB)	Page Header ItemId[]
	Page 1 (8 KB)	Tuple (red 1)
	Page 2 (8 KB)	Tuple (red 2)
orders.dat	Page 0 (8 KB)	Free Space
	Page 1 (8 KB)	Tuple Tuple

Unutar jedne stranice nalaze se:

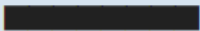
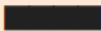
- page header (metapodaci o stranici),
- ItemId niz (pokazivači na položaj svakog torka unutar stranice),
- same torke (redovi podataka),
- slobodan prostor za nove unose.

Razumevanje stranica je ključno jer svi mehanizmi optimizacije – keširanje, indeksi, prefetching – rade na nivou stranica.

2.3 Disk i memorija – razlika u brzini

Razlika u brzini između RAM-a i diska jedan je od ključnih faktora koji utiču na performanse DBMS-a. Dok je pristup podacima u RAM memoriji brz i konstantan, pristup podacima na disku je znatno sporiji zbog fizičkih ograničenja medijuma.

HDD diskovi zahtevaju mehaničke operacije (seek time – pomeranje glave, rotational delay – čekanje na rotaciju diska) pre nego što se podaci mogu pročitati. SSD diskovi nemaju pokretnih delova, ali su i dalje višestruko sporiji od RAM-a. Upravo zbog ove razlike, minimizacija disk I/O operacija je primarni cilj svakog storage engine-a.

Nivo memorije	Tipična latencija	Relativna brzina
CPU registri / keš (L1)	~1 ns	 10.000×
RAM memorija	~100 ns	 1.000×
SSD disk (NVMe)	~100 μs	 10×
HDD disk (magnetni)	~10 ms	 1× (osnova)

Zbog ove razlike, DBMS sistemi pokušavaju da što veći deo podataka drže u RAM-u. U PostgreSQL-u, ovo se postiže kroz `shared_buffers` (buffer pool) i OS page cache. Svaka stranica koja se učitava sa diska ostaje u memoriji koliko god je moguće, sprečavajući ponovne skupe disk pristupe.

2.4 Disk I/O operacije

Disk I/O (input/output) operacije predstavljaju sve operacije čitanja i pisanja podataka između diska i RAM-a. U sistemima baza podataka, ovo su najskuplji deo izvršavanja upita i direktno određuju performanse sistema.

- **shared hit** u PostgreSQL terminologiji znači da je stranica pronađena u buffer pool-u (bez disk pristupa).
- **shared read** znači da je stranica morala da se učitava sa diska – svaki takav red u EXPLAIN ANALYZE izlazu predstavlja I/O trošak.

Svaka operacija čitanja sa diska podrazumeva učitavanje cele stranice (8 KB), bez obzira na to koliko bajtova je zapravo potrebno. Zbog toga I/O trošak se direktno meri brojem stranica, ne brojem redova.

2.5 Sekvencijalni i nasumični pristup podacima

Način na koji sistem baze podataka pristupa podacima sa diska ima veliki uticaj na performanse izvršavanja upita. U praksi se najčešće razlikuju dva modela pristupa: sekvencijalni (sequential access) i nasumični (random access).

Sekvencijalni pristup podrazumeva čitanje podataka redom, gde se više uzastopnih stranica učitava sa diska u kontinualnom toku. Ovaj način pristupa je efikasniji jer minimizuje broj skupih operacija pomeranja glave diska i omogućava bolje iskorišćenje propusnog opsega. Zbog toga operacije kao što je full table scan često koriste sekvencijalni pristup, posebno kada se obrađuje veći deo tabele.

Nasuprot tome, nasumični pristup podrazumeva čitanje podataka sa različitih, nepovezanih lokacija na disku. Svaki takav pristup može zahtevati posebnu disk I/O operaciju, što značajno povećava ukupno vreme izvršavanja. Nasumični pristup se najčešće javlja kod indeksiranih pretraga, gde sistem koristi indeks da direktno pristupi određenim stranicama koje sadrže tražene podatke.

Razlika između ova dva pristupa posebno dolazi do izražaja kod velikih tabela. Sekvencijalni pristup je efikasniji kada je potrebno obraditi veliki procenat podataka, dok je nasumični pristup bolji kada se pristupa malom broju specifičnih zapisa.

U sistemu kao što je PostgreSQL, optimizator upita bira između ova dva pristupa na osnovu procenjene selektivnosti upita i dostupnih struktura kao što su indeksi. Zbog toga razumevanje razlike između sekvencijalnog i nasumičnog pristupa predstavlja osnovu za analizu planova izvršavanja upita i performansi baze podataka.

2.6 Read/Write putanja podataka

Svaki pristup podacima u PostgreSQL-u prolazi kroz jasno definisan tok između diska, memorije i procesora. Ovaj tok se naziva read/write putanja i predstavlja osnovni mehanizam kroz koji storage engine izvršava operacije nad podacima. Razumevanje ove putanje je osnova za analizu performansi –

jer upravo ovde buffer pool ima ključnu ulogu.

Putanja za čitanje (SELECT): Kada executor zahteva određenu stranicu, buffer manager najpre proverava da li se ona već nalazi u buffer pool-u. Ako jeste (*cache hit*), podaci se čitaju direktno iz RAM-a bez ikakvog pristupa disku, što značajno ubrzava izvršavanje upita. Ako stranica nije prisutna (*cache miss*), sistem mora da izvrši I/O operaciju – stranica se učitava sa diska, smešta u buffer pool i tek onda prosleđuje executoru na obradu. Odnos između cache hit i cache miss situacija direktno određuje performanse sistema: što je veći procenat cache hit-ova, manji je broj sporih disk operacija.

Putanja za pisanje (INSERT/UPDATE/DELETE): Izmene se uvek vrše najpre u memoriji. Stranica koja je izmenjena, ali još nije upisana na disk, označava se kao *dirty page*. Pre nego što se dirty page upiše na disk, PostgreSQL obavezno upisuje odgovarajući zapis u WAL (*Write-Ahead Log*). Tek nakon toga, background writer proces asinhronično flush-uje dirty pages na disk u pozadini, bez blokiranja tekućih transakcija.

Razlog zašto se izmene ne upisuju odmah na disk jeste performansa. Direktno pisanje pri svakoj izmeni (*write-through*) bilo bi izuzetno sporo za write-intenzivne workload-ove, jer bi svaka INSERT ili UPDATE operacija čekala na fizički disk. Umesto toga, PostgreSQL koristi *write-back* strategiju: izmene se akumuliraju u buffer pool-u i periodično flush-uju u grupama, čime se značajno smanjuje broj disk operacija. Konzistentnost podataka se garantuje isključivo kroz WAL – pravilo je da log zapis mora biti fizički na disku pre same stranice podataka. Na taj način, u slučaju pada sistema, PostgreSQL može da ponovi sve committed izmene iz WAL-a, čak i ako dirty pages u tom trenutku nisu bile flush-ovane na disk.

Read/write putanja tako pokazuje da je buffer pool centralni posrednik između diska i procesora – njegova veličina i strategija upravljanja direktno određuju koliko često sistem mora da posegne za sporim disk operacijama, što ima ključan uticaj na ukupne performanse baze podataka.

3. Buffer pool

Buffer pool je centralna memorijska komponenta sistema za upravljanje bazama podataka (DBMS), smeštena između diska i procesora. Njegova osnovna uloga je keširanje stranica podataka u RAM memoriji kako bi se smanjio broj skupih pristupa disku i ubrzalo izvršavanje SQL upita. Bez buffer pool-a, svaki pristup podacima podrazumevao bi direktno čitanje sa diska – što bi, zbog razlike u brzini između RAM-a i diska, učinilo sistem višestruko sporijim.

Buffer pool (u PostgreSQL-u poznat kao `shared_buffers`) je memorijski prostor u okviru kojeg DBMS čuva kopije disk stranica. Svaka stranica ima veličinu 8 KB, a sve operacije čitanja i pisanja prolaze kroz ovaj sloj pre nego što dospeju do diska ili procesora.

3.1 Osnovni koncepti buffer pool-a

Buffer pool funkcioniše na principu lokalnosti podataka – pretpostavlja da će stranice kojima se nedavno pristupalo biti potrebne i u narednim operacijama. Tri osnovna koncepta koja definišu njegovo ponašanje su keširanje stranica, smanjenje I/O operacija i upravljanje izmenjenim (*dirty*) stranicama.

3.1.1 Keširanje stranica

Keširanje (*caching*) podrazumeva čuvanje kopija disk stranica u RAM memoriji. Kada executor zahteva određenu stranicu, buffer manager najpre proverava da li se ona već nalazi u buffer pool-u.

Ako jeste, stranica se čita direktno iz memorije bez ikakve disk operacije – ovo se naziva cache hit. Ako stranica nije prisutna, sistem mora da je učita sa diska (cache miss), smesti u slobodan frame i tek tada prosledi executoru. PostgreSQL čuva stranice u memorijskom prostoru definisanom parametrom `shared_buffers`. Podrazumevana vrednost u novijim verzijama je 128 MB, ali se u produkcijskim okruženjima preporučuje podešavanje na oko 25% ukupnog RAM-a sistema.

3.1.2 Smanjenje I/O operacija

Disk I/O operacije su najskuplji deo izvršavanja upita – pristup HDD disku traje oko 10 ms, SSD disku oko 100 μ s, dok je pristup RAM-u oko 100 ns. To znači da je RAM 100 do 100.000 puta brži od diska, zavisno od tipa medijuma. Svaki cache hit koji zamenjuje disk pristup predstavlja direktno ubrzanje sistema. Efikasnost buffer pool-a meri se kroz buffer hit ratio – procenat zahteva koji su opsluženi iz memorije bez potrebe za diskom. U dobro podešenim sistemima ovaj procenat iznosi 95–99%.

3.1.3 Dirty pages

Dirty page (izmenjena stranica) je stranica koja je modifikovana u memoriji, ali još nije upisana na disk. Umesto trenutnog pisanja pri svakoj izmeni (write-through strategija), PostgreSQL koristi write-back pristup: izmene se akumuliraju u buffer pool-u i periodično se flush-uju na disk u grupama. Ova strategija značajno smanjuje broj disk operacija pri write-intenzivnim workload-ovima. Konzistentnost se garantuje kroz WAL (Write-Ahead Log) – svaki zapis mora biti u WAL-u pre nego što odgovarajuća dirty page stigne na disk. Na taj način, u slučaju pada sistema, sve committed izmene mogu biti rekonstruisane iz WAL-a.

U PostgreSQL-u, background writer i checkpoint procesi periodično flush-uju dirty stranice. Parametri `bgwriter_lru_maxpages` i `checkpoint_completion_target` kontrolišu intenzitet i raspored ovih operacija.

3.2 Cache hit i cache miss

Odnos između cache hit i cache miss situacija direktno određuje performanse sistema. Što je manji broj cache miss-ova, to je manji broj skupih disk operacija i kraće vreme izvršavanja upita.

- **Cache hit** - Tražena stranica je pronađena u buffer pool-u – podaci se čitaju direktno iz RAM-a, bez ikakve disk operacije. Pristup traje oko 100 ns.
- **Cache miss** - Tražena stranica nije u buffer pool-u – sistem mora da je učita sa diska. Pristup traje od 100 μ s (SSD) do 10 ms (HDD). DBMS mora da pronađe slobodan frame i izvrši I/O operaciju.
- **shared hit** - PostgreSQL termin za cache hit – stranica je bila u `shared_buffers`.
- **shared read** - PostgreSQL termin za cache miss – stranica je morala biti učitana sa diska.

U PostgreSQL izlazu EXPLAIN ANALYZE, vrednosti `shared hit` i `shared read` direktno pokazuju koliko stranica je opsluženo iz memorije, a koliko sa diska. Analiza ovih vrednosti je ključna za dijagnostiku performansi upita.

3.3 Upravljanje buffer pool-om

Pošto je memorijski prostor ograničen, buffer pool ne može da čuva sve stranice istovremeno. DBMS koristi različite strategije za upravljanje memorijom i odlučivanje koje stranice ostaju u kešu, a koje se uklanjaju – ovaj proces se naziva evikcija stranica.

3.3.1 Politike zamene stranica (evikcija)

Kada je buffer pool pun i potrebno je učitati novu stranicu, sistem mora izabrati stranicu koja će biti uklonjena. Redosled provera je važan kako bi se izbeglo uklanjanje stranica koje su trenutno aktivno u upotrebi ili koje imaju neupisane izmene.

Proces evikcije uvek prati sledeći redosled:

- Izbor kandidata prema politici zamene (LRU, CLOCK ili varijante)
- Provera pin counter-a – stranica koja se aktivno koristi ne može biti izbačena
- Provera dirty flag-a – ako je stranica dirty, mora se najpre upisati na disk (flush)
- Oslobađanje frame-a i smeštanje nove stranice

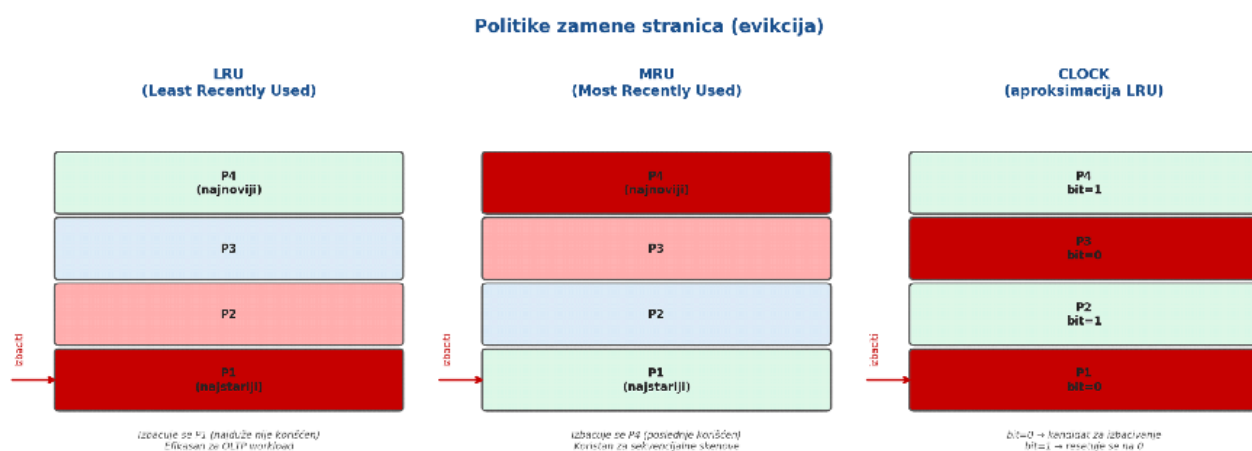
3.3.2 LRU, MRU i CLOCK algoritmi

Politike zamene stranica pokušavaju da predvide buduću upotrebu podataka na osnovu prethodnih pristupa. Svaka politika donosi kompromis između tačnosti predikcije i složenosti implementacije.

LRU (Least Recently Used) uklanja stranicu koja se najduže nije koristila. Zasniva se na principu vremensko-prostorne lokalnosti i efikasan je u OLTP sistemima gde su pristupi ponavljajući. Mana LRU algoritma je tzv. cache pollution – pri sekvencijalnom skenu velike tabele, LRU puni bafer stranicama koje će verovatno biti korišćene samo jednom, potiskujući korisne stranice.

MRU (Most Recently Used) radi suprotno – uklanja stranicu koja je poslednje korišćena. Ovo je korisno pri sekvencijalnim skenovima, gde se podaci koriste jednom i neće biti ponovo potrebni. MRU je retko podrazumevana politika, ali se koristi u specifičnim scenarijima.

CLOCK algoritam je optimizovana aproksimacija LRU-a. Svakoj stranici dodeljuje se referentni bit koji se postavlja na 1 pri svakom pristupu. Stranice su organizovane u kružnu strukturu; kazaljka prolazi kroz njih i pri nailasku: ako je bit 1, resetuje ga; ako je bit 0, stranica se uklanja. CLOCK postiže dobar balans između performansi i jednostavnosti implementacije.



3.3.3 Free list i midpoint insertion

Free list je lista slobodnih frame-ova koji su odmah dostupni za smeštanje novih stranica bez potrebe za evikcijom. Kada buffer pool nije pun, nove stranice se smeštaju direktno u slobodne frame-ove. Midpoint insertion strategija (koristi je InnoDB u MySQL-u) ubacuje nove stranice u sredinu LRU liste umesto na njen vrh. Na taj način, jednokratno korišćene stranice (npr. pri sekvencijalnom skenu) brzo

tonu ka repu liste i bivaju izbačene, ne potiskujući stranice koje se zaista često koriste.

3.3.4 OS page cache sloj

Pored DBMS buffer pool-a, operativni sistem koristi sopstveni keš (OS page cache) koji čuva disk blokove u memoriji. Ovaj sloj može dodatno smanjiti broj fizičkih disk operacija. Međutim, u slučaju PostgreSQL-a, dolazi do dupliranja keširanja – iste stranice mogu biti i u `shared_buffers` i u OS page cache-u, što nepotrebno troši RAM. Zbog toga neki DBMS sistemi koriste direktan I/O (`O_DIRECT` zastava) koji zaobilazi OS page cache. PostgreSQL to ne radi podrazumevano, ali se efekat može kontrolisati kroz odgovarajuće OS podešavanja.

3.4 Arhitektura buffer pool-a

Buffer pool je interno organizovan kao skup memorijskih frame-ova koji sadrže stranice učitane sa diska. Ključna interna struktura koja povezuje identifikatore stranica sa fizičkim frame-ovima jeste page table.

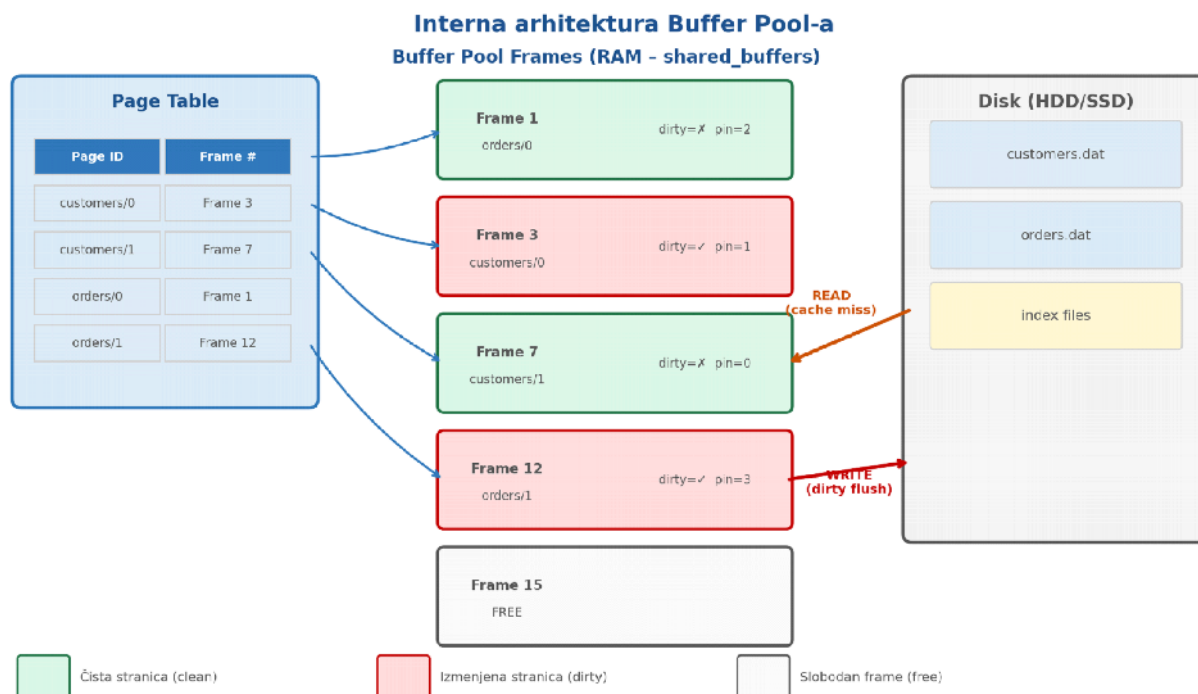
3.4.1 Interna organizacija – page table i frame metapodaci

Page table je heš tabela koja mapira identifikator stranice ($\text{page ID} = \text{relfilenode} + \text{block number}$) na odgovarajući frame u memoriji. Zahvaljujući ovoj strukturi, buffer manager može u konstantnom vremenu $O(1)$ proveriti da li se stranica nalazi u kešu.

Svaki frame u buffer pool-u nosi sledeće metapodatke:

- Page ID (tag) – identifikator stranice koja je smeštena u frame-u
- Dirty flag – označava da li je stranica izmenjena, ali još nije upisana na disk
- Pin counter (reference count) – broj procesa koji trenutno koriste stranicu; stranica sa $\text{pin} > 0$ ne može biti evict-ovana
- Usage count – broj pristupa, koristi se za CLOCK i slične algoritme

Princip rada je sledeći: kada executor zahteva stranicu, buffer manager izračuna heš ključ, pretražuje page table i pronalazi odgovarajući frame (cache hit) ili utvrđuje da stranica nije u memoriji (cache miss). U slučaju miss-a, pronalazi slobodan ili evict-uje postojeći frame, učitava stranicu sa diska i unosi je u page table.



3.4.2 Buffer Pool Extension (BPE)

Buffer Pool Extension (BPE) je koncept proširenja efektivnog kapaciteta memorijskog keša korišćenjem sporijeg, ali većeg medijuma – najčešće SSD diska. Stranice koje ne mogu da stanu u RAM premešta se na sekundarni sloj, čime se smanjuje broj pristupa HDD-u.

Microsoft SQL Server implementira BPE kao eksplicitnu funkcionalnost. PostgreSQL nema ekvivalentni mehanizam, ali sličan efekat postiže kombinacijom OS page cache-a i SSD swap prostora. Alternativno, konfiguirisanje tablespace-ova na NVMe diskovima može da imitira višeslojni keš.

3.4.3 Konkurentnost i latch mehanizmi

U produkcijskim sistemima, više procesa i niti istovremeno pristupa buffer pool-u. Da bi se sprečila oštećenja internih struktura, koriste se dva nivoa sinhronizacije:

1. Lock - Logički nivo zaštite. Štiti podatke (redove, tabele, stranice) tokom transakcija. Ima semantiku trajanja transakcije i podržava čekanje, eskalaciju i detekciju deadlock-a.
2. Latch- Fizički nivo zaštite. Štiti interne memorijske strukture (buffer pool, page table, hash lanac). Kratkotrajan – drži se samo dok se operacija izvodi. Nema detekciju deadlock-a; mora se pažljivo redosledom koristiti.
3. Buffer content lock - Deli se na shared (više čitalaca istovremeno) i exclusive (samo jedan pisac). Čuvar konzistentnosti sadržaja stranice dok je executor koristi.

Latch contention (sukobi pri zaključavanju) može biti ozbiljan uzrok degradacije performansi u sistemima sa visokim stepenom paralelizma. PostgreSQL koristi particionisanje buffer pool-a na više nezavisnih latch zona kako bi smanjio ovu vrstu konflikata.

3.4.4 Uticaj na performanse

Arhitektura buffer pool-a direktno određuje performanse sistema u svim dimenzijama. Efikasno keširanje smanjuje disk I/O, pravilna politika evikcije osigurava da u memoriji ostaju relevantne stranice, a dobro dizajniran latch mehanizam omogućava visoki stepen paralelizma bez narušavanja

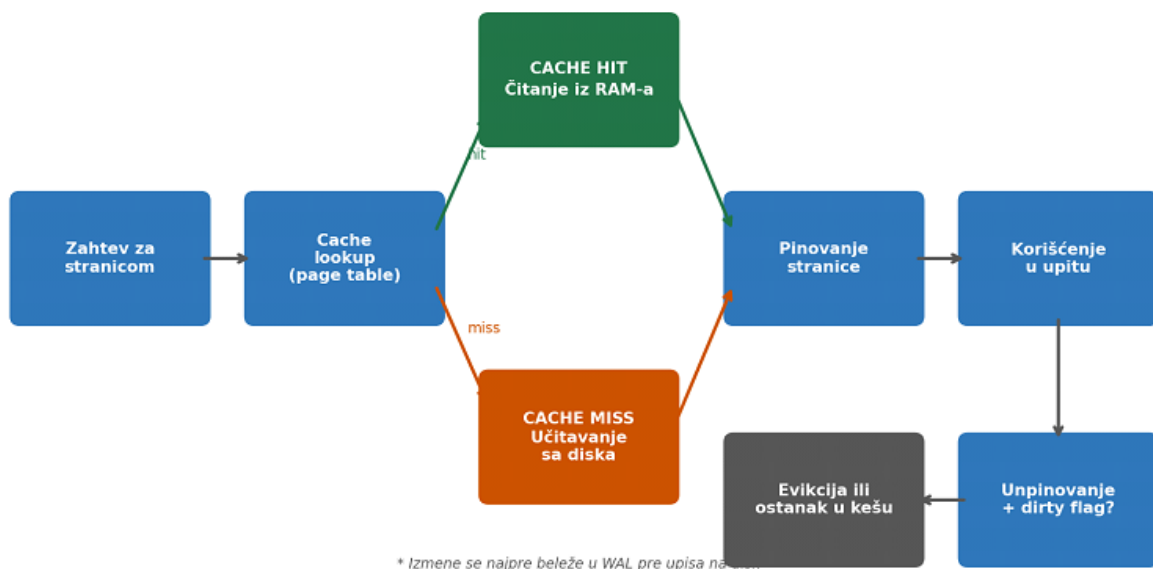
konzistentnosti.

Razlika između cold cache i warm cache scenarija može biti i do nekoliko desetina puta. U warm cache stanju, pri ponavljanju istog upita, vreme se stabilizuje na minimum jer se sve operacije svode na CPU procesiranje bez I/O opterećenja.

3.5 Životni ciklus stranice u buffer pool-u

Svaka stranica prolazi kroz jasno definisan životni ciklus od trenutka kada je zatražena od executora do eventualnog uklanjanja iz memorije ili upisa na disk. Razumevanje ovog ciklusa je osnova za analizu performansi i dijagnostiku uskih grla u DBMS-u.

Životni ciklus stranice u Buffer Pool-u



Ciklus obuhvata sledeće faze:

- Zahtev za stranicom – executor šalje zahtev buffer manageru
- Cache lookup – buffer manager pretražuje page table (heš tabelu)
- Cache hit – stranica je u RAM-u; pinuje se i direktno prosleđuje executoru
- Cache miss – stranica nije u RAM-u; mora se učitati sa diska (I/O operacija)
- Pinovanje – pin counter se povećava; stranica ne može biti izbačena dok je u upotrebi
- Korišćenje u upitu – executor čita ili modifikuje sadržaj stranice
- Dirty marking – ako je stranica izmenjena, dirty flag se postavlja na true
- Unpinovanje – pin counter se smanjuje kada executor završi rad sa stranicom
- Evikcija ili ostanak – stranica ostaje u kešu za buduće upite ili se uklanja prema politici evikcije; dirty stranice se pre uklanjanja flush-uju na disk

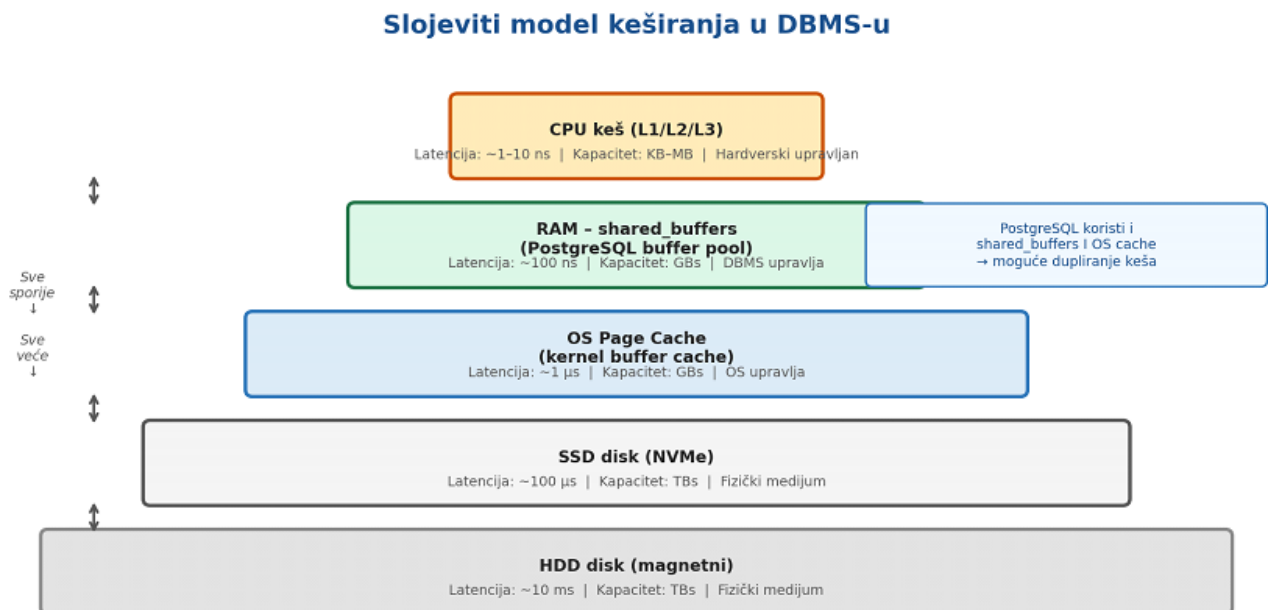
WAL garancija: Pre nego što se dirty page upiše na disk, PostgreSQL obavezno beleži izmenu u Write-Ahead Log (WAL). Ovim se garantuje da u slučaju pada sistema, sve committed izmene mogu biti rekonstruisane, čak i ako dirty pages u tom trenutku nisu bile fizički na disku.

4. Caching mehanizam i uticaj na performanse

Caching mehanizmi u sistemima baza podataka čine skup tehnika kojima se privremeno čuvaju kopije podataka u bržim slojevima memorijske hijerarhije, sa ciljem smanjenja broja skupih disk I/O operacija. Dok buffer pool (poglavlje 4) predstavlja primarni DBMS-ov keš, PostgreSQL sistem u celini oslanja se na nekoliko međusobno komplementarnih slojeva keširanja – svaki sa drugačijim kapacitetom, latencijom i načinom upravljanja.

4.1 Slojevit memorijski hijerarhija

Razumevanje caching mehanizama počinje od memorijske hijerarhije sistema. Svaki nivo hijerarhije nudi drugačiji kompromis između brzine pristupa, kapaciteta i cene. DBMS pokušava da drži što više podataka na višim, bržim nivoima kako bi minimizovao potrebu za pristupom nižim, sporijim slojevima.



Ključna karakteristika ove hijerarhije je da nijedan sloj ne funkcioniše nezavisno – podatak koji stigne do procesora mora proći kroz sve niže nivoe, a svaki sloj koji ga uhvati (kešira) sprečava ponavljanje tog putovanja prema disku pri sledećem pristupu.

- CPU keš (L1/L2/L3) - hardverski upravljan, nevidljiv za DBMS. Latencija 1–10 ns. Kešira instrukcije i nedavno korišćene delove radnih struktura (npr. indeksne čvorove).
- shared_buffers - primarni PostgreSQL buffer pool. Drži stranice podataka (8 KB svaka). Konfigurišu se parametrom shared_buffers u postgresql.conf. Preporučena vrednost: ~25% RAM-a.
- OS page cache - kernel-level keš koji čuva disk blokove u RAM-u. PostgreSQL ga koristi posredno – postoji rizik dupliranja stranica (i u shared_buffers i u OS cache-u).
- SSD / NVMe - fizički medijum. Latencija ~100 μs. PostgreSQL može koristiti tablespace-ove na SSD-u za vruće tabele i indekse.

- HDD - najsporiji nivo. Latencija ~10 ms. Sekvencijalni pristup znatno bolji od nasumičnog zbog mehaničkih ograničenja.

4.2 Cold cache i warm cache

Stanje buffer pool-a u trenutku izvršavanja upita direktno određuje da li će sistem moći da opsluži zahtev iz memorije ili mora da posegne za diskom. Razlikuju se dva karakteristična stanja:

- Cold cache - Buffer pool je prazan ili ne sadrži relevantne stranice. Javlja se pri prvom pokretanju sistema, nakon DISCARD ALL komande, ili pri prvom pristupu novim podacima. Svaki zahtev generiše Shared Read (disk I/O), što rezultuje visokim vremenima izvršavanja.
- Warm cache - Relevantne stranice su već prisutne u buffer pool-u iz prethodnih upita. Zahtevi se opslužuju kao Shared Hit (iz RAM-a). Vreme izvršavanja se stabilizuje na minimum, a sistem radi isključivo CPU operacijama bez I/O.
- Tepid cache - Prelazno stanje – deo stranica je u kešu, deo ne. Tipično nakon restarta sistema ili pri proširenju skupa podataka. EXPLAIN ANALYZE će pokazati mešavinu shared hit i shared read vrednosti.

4.3 Caching mehanizmi pored buffer pool-a

Pored buffer pool-a, PostgreSQL koristi i nekoliko dopunskih mehanizama koji zajedno doprinose ukupnoj efikasnosti keširanja.

4.3.1 Plan cache (keš planova izvršavanja)

Plan cache čuva kompajlirane planove izvršavanja upita koji su prethodno optimizovani. Kada se koriste pripremljeni upiti (PREPARE / EXECUTE), PostgreSQL može ponovo iskoristiti plan bez ponovnog pokretanja optimizatora, čime se eliminiše vreme planiranja (planning time) pri svakom izvršavanju.

Iz praktičnih testova u ovom radu, planning time je iznosio 0.1–2.2 ms po upitu. Na sistemima sa visokim brojem kratkih upita (OLTP), akumulirani trošak planiranja može biti značajan – plan cache direktno eliminiše taj overhead.

Važno ograničenje: plan cache ne ažurira planove automatski kada se statistike podataka promene. Nakon ANALYZE ili masovnih izmena podataka, pripremljeni planovi mogu biti suboptimalni sve do ponovnog pripremanja.

4.3.2 OS page cache i interakcija sa shared_buffers

Operativni sistem autonomno kešira sve disk operacije kroz kernel page cache. Kada PostgreSQL pročita stranicu sa diska, OS je smešta u page cache; pri sledećem pristupu OS može opsluži zahtev bez fizičkog čitanja sa diska, čak i ako ta stranica nije u shared_buffers.

Ovo stvara efekat dvoslojan keširanja:

- Ako je stranica u shared_buffers → DBMS je opsluži direktno (najbrže)
- Ako je stranica samo u OS page cache → OS je prosledi bez diska, ali PostgreSQL mora da je smesti i u shared_buffers
- Ako nije ni u jednom kešu → fizički disk pristup (najsporije)

Ovo dupliranje može biti i prednost i mana: prednost je što OS cache deluje kao "besplatni" dodatni

sloj zaštite od disk pristupa; mana je da iste stranice zauzimaju duplo RAM-a. Zato se u određenim konfiguracijama preporučuje smanjenje `shared_buffers` i oslanjanje na OS page cache, ili obrnuto.

4.3.3 pg_prewarm – predgrevanje buffer pool-a

`pg_prewarm` je PostgreSQL ekstenzija koja omogućava eksplicitno učitavanje stranica u buffer pool pre nego što korisnici počnu da izvršavaju upite. Ovo je posebno korisno nakon restarta sistema, kada je keš potpuno hladan (cold cache), a operatori znaju koji podaci će biti odmah potrebni.

Primer korišćenja:

```
SELECT pg_prewarm('test_cache'); -- učitava celu tabelu
```

```
SELECT pg_prewarm('idx_test_cache_value'); -- učitava indeks
```

Bez `pg_prewarm`, sistem bi morao da "sam nauči" koje stranice su potrebne kroz prve realne upite – što znači da će ti rani upiti biti sporiji. Sa `pg_prewarm`, sistem startuje već u warm cache stanju za kritične objekte.

4.4 Efekat ponovljenih upita i buffer hit ratio

Jedan od najuočljivijih efekata caching mehanizama je dramatično ubrzanje ponovljenih upita. Ovo se najjasnije vidi kroz buffer hit ratio – ključnu metriku koja govori koliki procenat zahteva za stranicama je opslužen iz memorije.

Buffer Hit Ratio = $\text{shared_hit} / (\text{shared_hit} + \text{shared_read}) \times 100\%$

Iz praktičnih testova sprovedenih u ovom radu (tabela `test_cache`, 500.000 redova):

Scenario	shared_hit
SEQ scan – cold cache	~0
SEQ scan – warm cache	5682
Bitmap scan – 1. izvršavanje	506
Bitmap scan – 2. izvršavanje	510
Index Only Scan – warm	446

Tabela potvrđuje direktnu vezu između hit ratio-a i vremena izvršavanja. Čak i razlika između 99.2% i 100% hit ratio-a (svega 4 stranice čitane sa diska) utiče na vreme izvršavanja – bitmap scan se ubrzao sa 1.56 ms na 0.68 ms između prvog i drugog izvršavanja.

Praćenje hit ratio-a u PostgreSQL-u: Metriku buffer hit ratio-a moguće je pratiti u realnom vremenu kroz `pg_stat_bgwriter` i `pg_statio_user_tables` systemske pogleda. Za ukupni sistem, vrednost ispod 90% obično signalizira potrebu za povećanjem `shared_buffers` ili optimizacijom upita.

4.5 Povezanost sa performansama sistema

Caching mehanizmi nisu izolovana optimizacija – oni deluju u sprezi sa svim ostalim komponentama sistema i njihov uticaj na performanse zavisi od karakteristika radnog opterećenja (workload).

4.5.1 OLTP vs OLAP workload

Karakteristike keširanja razlikuju se prema tipu opterećenja:

- OLTP (transakcioni sistemi) – mali, česti upiti koji pristupaju istim redovima i stranicama.

Visok hit ratio je lako dostižan jer je radni skup podataka mali i lako staje u `shared_buffers`. LRU politika evikcije dobro funkcioniše u ovom scenariju.

- OLAP (analitički sistemi) – retki, obimni upiti koji skeniraju veliku količinu podataka. Teže je postići visok hit ratio jer radni skup može biti veći od dostupne memorije. Sekvencijalni skenovi mogu uzrokovati cache pollution i potiskivati korisne stranice.

4.5.2 Keširanje i optimizator upita

Optimizator upita u PostgreSQL-u u svojoj analizi troškova uzima u obzir i efikasnost keširanja. Parametar `effective_cache_size` govori optimizatoru koliko memorije (uključujući OS page cache) je efektivno dostupno za keširanje – veća vrednost podstiče optimizator da preferira index scan nad seq scan, jer pretpostavlja da će više podataka biti u memoriji.

Pravilno podešavanje `effective_cache_size` (tipično 50–75% ukupnog RAM-a) može direktno uticati na to koji plan izvršavanja optimizator izabere, čak i bez fizičke promene količine dostupne memorije.

4.5.3 Caching i konkurentni pristup

U višekorisničkim sistemima, konkurentni upiti dele isti buffer pool. To znači da upiti koji donose podatke u keš posredno pomažu i drugim upitima koji pristupaju istim stranicama – ovo je pozitivan efekat keš-topline (cache warming) koji se akumulira tokom rada sistema.

S druge strane, veliki analitički upit koji skenira čitavu tabelu može "istisnuti" stranice koje su koristili OLTP upiti. Upravo zbog toga PostgreSQL uvodi ring buffer strategiju za sekvencijalne skenove – ograničeni kružni bafer sprečava da seq scan zauzme ceo buffer pool i zagadi keš za ostatak sistema.

5. Prakticni primeri

5.1 Testiranje manje baze

5.1.1 Okruženje i konfiguracija

Sva testiranja su izvršena na PostgreSQL 18, korišćenjem alata pgAdmin. Testirana je tabela `test_cache` sa 500.000 redova, a baza podataka zauzima 67MB uz podešavanje `shared_buffers` = 128MB.

Budući da je baza manja od veličine buffer pool-a, PostgreSQL je u mogućnosti da kešira kompletne podatke tabele u memoriji. Ovo direktno objašnjava rezultate testiranja — nakon prvog učitavanja (cold cache), sva naredna izvršavanja koriste isključivo podatke iz memorije, bez disk I/O operacija.

Parametar	Vrednost
SUBP	PostgreSQL 18
Tabela	<code>test_cache</code>
Broj redova	500.000
Veličina baze	67 MB
<code>shared_buffers</code>	128 MB
Resetovanje keša	DISCARD ALL

Kreiranje

tabele:

```
Query Query History AI Assistant
1 CREATE TABLE test_cache (
2     id BIGSERIAL PRIMARY KEY,
3     value INT,
4     payload TEXT
5 );
```

Kreiranje

500000

random

vrednosti:

```
postgres/postgres@PostgreSQL 18
Query Query History AI Assistant
1 INSERT INTO test_cache (value, payload)
2 SELECT
3     (random() * 1000)::int,
4     repeat('x', 50)
5 FROM generate_series(1, 500000);
```

5.1.2 Sekvencijalno skeniranje bez indeksa (Seq Scan)

5.1.3.1 Prvo izvršavanje — cold cache

The screenshot shows a PostgreSQL query editor interface. At the top, the connection is 'postgres/postgres@PostgreSQL 18'. Below the connection bar is a toolbar with various icons for file operations, query execution, and settings. The 'Query' tab is active, showing a SQL query:

```
1 EXPLAIN ANALYZE
2 SELECT * FROM test_cache WHERE value = 500;
```

Below the query editor, the 'Data Output' tab is active, displaying the 'QUERY PLAN' for the executed query. The plan is shown in a table format with 12 rows of details.

	QUERY PLAN
1	Bitmap Heap Scan on test_cache (cost=8.26..1502.43 rows=495 width=63) (actual time=0.388..1.465 rows=535.00 loops=1)
2	Recheck Cond: (value = 500)
3	Heap Blocks: exact=506
4	Buffers: shared hit=506 read=4
5	-> Bitmap Index Scan on idx_test_cache_value (cost=0.00..8.14 rows=495 width=0) (actual time=0.237..0.237 rows=535.00 loops=1)
6	Index Cond: (value = 500)
7	Index Searches: 1
8	Buffers: shared read=4
9	Planning:
10	Buffers: shared hit=15 read=1 dirtied=2
11	Planning Time: 2.223 ms
12	Execution Time: 1.559 ms

PostgreSQL je koristio Bitmap Index Scan za identifikaciju 535 relevantnih redova, a zatim Bitmap Heap Scan za grupisano čitanje iz heap-a. Indeks je dramatično smanjio broj čitanih stranica sa 5.682 na svega 506. Mali broj disk čitanja (shared read = 4) ukazuje da indeksne stranice nisu bile u potpunosti keširane — ovo je cold cache momenat.

5.1.3.2 Drugo izvršavanje — warm cache

postgres/postgres@PostgreSQL 18

Query Query History AI Assistant

```

1 EXPLAIN ANALYZE
2 SELECT * FROM test_cache WHERE value = 500;

```

Data Output Messages Notifications

Showing rows: 1 to 10 Page No: 1

	QUERY PLAN text
1	Bitmap Heap Scan on test_cache (cost=8.26..1502.43 rows=495 width=63) (actual time=0.134..0.640 rows=535.00 loops=1)
2	Recheck Cond: (value = 500)
3	Heap Blocks: exact=506
4	Buffers: shared hit=510
5	-> Bitmap Index Scan on idx_test_cache_value (cost=0.00..8.14 rows=495 width=0) (actual time=0.076..0.076 rows=535.00 loo...)
6	Index Cond: (value = 500)
7	Index Searches: 1
8	Buffers: shared hit=4
9	Planning Time: 0.120 ms
10	Execution Time: 0.676 ms

Ponovnim izvršavanjem istog upita, sve relevantne stranice su već bile prisutne u buffer pool-u. Plan izvršavanja ostao je identičan, ali je vreme prepolovljeno — sa 1.559 ms na 0.676 ms. Razlika je isključivo posledica stanja keša, a ne promene upita ili plana. Ovo jasno pokazuje da buffer pool direktno utiče na performanse, dok optimizator ne menja odluku na osnovu ponavljanja upita.

5.1.4 Poređenje: Seq Scan vs Index Scan (cold vs warm)

	Seq Scan	Index — cold	Index — warm
Vreme izvršavanja	48.926 ms	1.559 ms	0.676 ms
Stranica čitano	5.682	510	510
Shared hit	5.682	506	510
Shared read (disk)	0	4	0
Plan	Parallel Seq Scan	Bitmap Scan	Bitmap Scan

5.1.5 Uticaj selektivnosti na izbor plana izvršavanja

postgres/postgres@PostgreSQL 18

No limit

Query Query History AI Assistant Scratch Pad ✕

```

1 EXPLAIN ANALYZE
2 SELECT * FROM test_cache WHERE value < 200;
```

Data Output Messages Notifications

Showing rows: 1 to 10 Page No: 1 of 1

	QUERY PLAN text
1	Bitmap Heap Scan on test_cache (cost=1135.21..8066.88 rows=99973 width=63) (actual time=7.268..23.902 rows=99357.00 loops=1)
2	Recheck Cond: (value < 200)
3	Heap Blocks: exact=5682
4	Buffers: shared hit=5706 read=66
5	-> Bitmap Index Scan on idx_test_cache_value (cost=0.00..1110.22 rows=99973 width=0) (actual time=6.471..6.471 rows=99357.00 loo...)
6	Index Cond: (value < 200)
7	Index Searches: 1
8	Buffers: shared hit=24 read=66
9	Planning Time: 0.144 ms
10	Execution Time: 27.301 ms

Za upit value < 200, koji vraća ~20% tabele, PostgreSQL je koristio Bitmap Index Scan jer indeksni pristup još uvek donosi korist grupisanim čitanjem heap blokova. Za upit value < 800, koji vraća ~80% tabele, optimizir je procenio da bi indeksni pristup generisao prevelik broj nasumičnih heap pristupa i prešao na linearno Seq Scan čitanje.

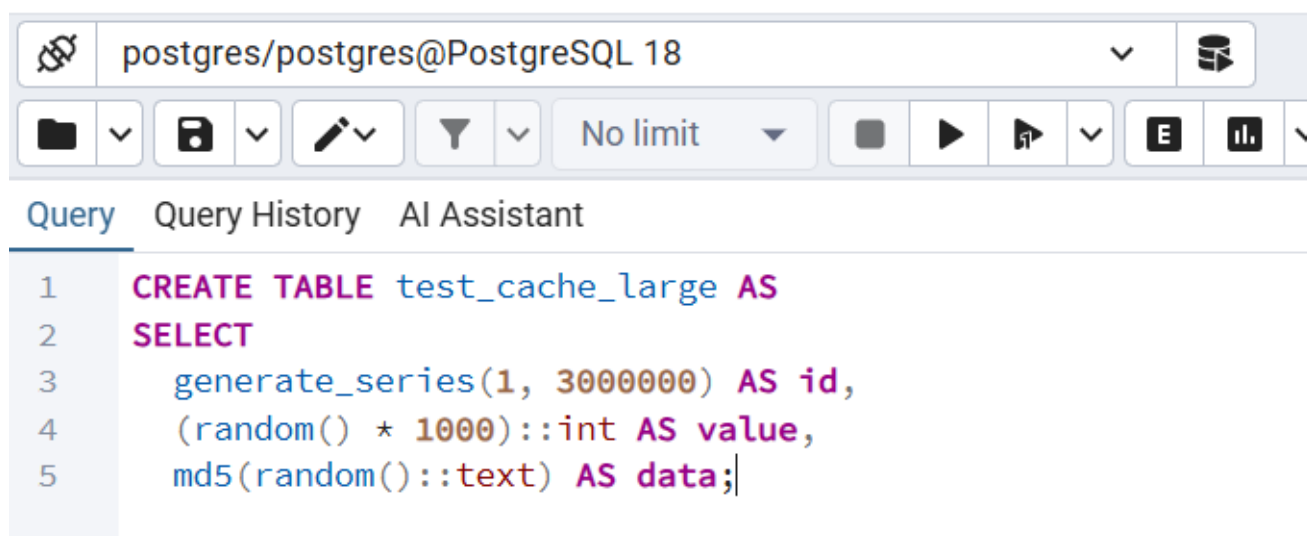
Interesantno je da value < 200 (Bitmap Scan) ima veći shared read (66) nego value < 800 (Seq Scan, shared read = 0). Razlog je što Bitmap Scan pristupa nasumičnim blokovima raspoređenim po celoj tabeli, dok Seq Scan čita linearno i pronalazi deo podataka već keširanih iz prethodnih testova. Ovo pokazuje da izbor plana i stanje keša zajedno određuju finalne performanse.

5.2 Testiranje veće baze

5.2.1 Okruženje i konfiguracija

Sva testiranja su izvršena na PostgreSQL 18, korišćenjem alata pgAdmin. Testirana je tabela test_cache sa 3000000 redova, a baza podataka zauzima 287MB uz podešavanje shared_buffers = 128MB.

Kreiranje tabele:



The screenshot shows the pgAdmin interface for PostgreSQL 18. The top toolbar includes icons for connecting, saving, editing, filtering, and executing queries. Below the toolbar, the 'Query' tab is active, displaying a SQL script to create a table named test_cache_large. The script uses the generate_series function to create 3,000,000 rows, each with a unique ID, a random integer value, and a random text data field.

```
1 CREATE TABLE test_cache_large AS
2 SELECT
3     generate_series(1, 3000000) AS id,
4     (random() * 1000)::int AS value,
5     md5(random()::text) AS data;
```

5.2.2 Seq Scan bez indeksa

postgres/postgres@PostgreSQL 18

Query Query History AI Assistant Scratch Pad

```

1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT * FROM test_cache_large WHERE value = 500;

```

Data Output Messages Notifications

Showing rows: 1 to 12 Page No: 1

	QUERY PLAN
1	Gather (cost=1000.00..45019.00 rows=2980 width=41) (actual time=1.079..203.248 rows=3081.00 loops=1)
2	Workers Planned: 2
3	Workers Launched: 2
4	Buffers: shared hit=2304 read=25792
5	-> Parallel Seq Scan on test_cache_large (cost=0.00..43721.00 rows=1242 width=41) (actual time=2.198..164.362 rows=1027.00 loo...
6	Filter: (value = 500)
7	Rows Removed by Filter: 998973
8	Buffers: shared hit=2304 read=25792
9	Planning:
10	Buffers: shared hit=13 dirtied=1
11	Planning Time: 1.316 ms
12	Execution Time: 203.542 ms

PostgreSQL je koristio paralelno sekvencijalno skeniranje sa dva workera, čitajući celu tabelu stranica po stranicu. Uprkos tome što je planirano oko 29.800 redova, izvršeno je paralelno skeniranje preko svih stranica tabele. PostgreSQL je iskoristio 2.304 buffera iz shared cache-a, ali je morao da učita čak 25.792 stranice sa diska (shared read=25.792). Paralelizacija je pomogla u raspodeli posla između workera, ali je sekvencijalno čitanje cele tabele rezultiralo znatno dužim izvršavanjem — execution time od 203.542 ms.

5.2.3 Kreiranje indeksa, pa opet:

```
CREATE INDEX idx_large_value ON test_cache_large(value);
```

5.2.3.1 Prvo izvršavanje — cold cache

Query
Query History
AI Assistant
Scratch

1
2

EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM test_cache_large WHERE value = 500;

Data Output
Messages
Notifications

Showing rows: 1 to 12
Page No: 1

QUERY PLAN	
	text
1	Bitmap Heap Scan on test_cache_large (cost=35.52..8698.27 rows=2980 width=41) (actual time=0.823..19.805 rows=3081.00 ...
2	Recheck Cond: (value = 500)
3	Heap Blocks: exact=2914
4	Buffers: shared hit=287 read=2632
5	-> Bitmap Index Scan on idx_large_value (cost=0.00..34.78 rows=2980 width=0) (actual time=0.396..0.396 rows=3081.00 loo...
6	Index Cond: (value = 500)
7	Index Searches: 1
8	Buffers: shared read=5
9	Planning:
10	Buffers: shared hit=15 read=1
11	Planning Time: 1.547 ms
12	Execution Time: 20.160 ms

PostgreSQL je koristio Bitmap Index Scan za identifikaciju 3.081 relevantnih redova, a zatim Bitmap Heap Scan za grupisano čitanje iz heap-a. Indeks je dramatično smanjio broj čitanih stranica. Indeksne stranice su zahtevale samo 5 disk čitanja (shared read=5), dok je Heap pristup zahtevao 2.632 disk čitanja (shared hit=287, read=2632). Execution time je pao na 20.160 ms — približno 10x brže u odnosu na paralelno sekvencijalno skeniranje.

5.2.3.2 Drugo izvršavanje — warm cache

The screenshot shows a PostgreSQL query editor interface. At the top, the database connection is 'postgres/postgres@PostgreSQL 18'. Below the connection bar is a toolbar with various icons for file operations, query execution, and settings. The 'Query' tab is active, showing the following SQL query:

```
1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT * FROM test_cache_large WHERE value = 500;
```

Below the query editor, the 'Data Output' tab is active, displaying the query plan. The plan shows a Bitmap Index Scan on 'idx_large_value' followed by a Bitmap Heap Scan on 'test_cache_large'. The execution time is 3.157 ms.

Step	Operation
1	Bitmap Heap Scan on test_cache_large (cost=35.52..8698.27 rows=2980 width=41) (actual time=0.963..2.967 rows=3081.00 l...
2	Recheck Cond: (value = 500)
3	Heap Blocks: exact=2914
4	Buffers: shared hit=2919
5	-> Bitmap Index Scan on idx_large_value (cost=0.00..34.78 rows=2980 width=0) (actual time=0.531..0.531 rows=3081.00 loo...
6	Index Cond: (value = 500)
7	Index Searches: 1
8	Buffers: shared hit=5
9	Planning Time: 0.116 ms
10	Execution Time: 3.157 ms

PostgreSQL je koristio Bitmap Index Scan za identifikaciju 3.081 relevantnih redova, a zatim Bitmap Heap Scan za pristup podacima u heap-u. Zahvaljujući toplom kešu, gotovo sve potrebne stranice su već bile prisutne u buffer pool-u. Indeks je očitao iz keša (shared hit=5), dok je heap pristup ostvario čak 2.919 shared hit-ova uz nula disk čitanja.

Execution time je pao na samo **3.157 ms**, što je višestruko brže u odnosu na cold cache izvršavanje. Ovo jasno demonstrira koliki uticaj ima stanje buffer pool-a na performanse indeksnog pristupa.

5.2.4 Uticaj selektivnosti na izbor plana izvršavanja

postgres/postgres@PostgreSQL 18

Query Query History AI Assistant

```

1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT * FROM test_cache_large WHERE value < 200;

```

Scratch Pad

Data Output Messages Notifications

Showing rows: 1 to 10 Page No: 1 of

	QUERY PLAN
1	Bitmap Heap Scan on test_cache_large (cost=6657.07..42173.46 rows=593631 width=41) (actual time=42.033..224.359 rows=597622.0...
2	Recheck Cond: (value < 200)
3	Heap Blocks: exact=28038
4	Buffers: shared hit=702 read=27852 written=15
5	-> Bitmap Index Scan on idx_large_value (cost=0.00..6508.66 rows=593631 width=0) (actual time=36.161..36.161 rows=597622.00 loo...
6	Index Cond: (value < 200)
7	Index Searches: 1
8	Buffers: shared hit=1 read=515
9	Planning Time: 0.145 ms
10	Execution Time: 243.785 ms

postgres/postgres@PostgreSQL 18

Query Query History AI Assistant

```

1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT * FROM test_cache_large WHERE value < 800;

```

Scratch Pad

Data Output Messages Notifications

Showing rows: 1 to 6 Page No: 1 of

	QUERY PLAN
1	Seq Scan on test_cache_large (cost=0.00..65596.00 rows=2399411 width=41) (actual time=0.130..794.091 rows=2398253.00 loo...
2	Filter: (value < 800)
3	Rows Removed by Filter: 601747
4	Buffers: shared hit=16210 read=11886
5	Planning Time: 0.146 ms
6	Execution Time: 918.373 ms

Za upit `value < 200`, koji vraća približno 20% tabele, PostgreSQL je i dalje izabrao Bitmap Index Scan + Bitmap Heap Scan. Indeksni pristup je još uvek donosio korist zahvaljujući grupisanom čitanju heap blokova. U ovom slučaju je broj disk čitanja bio veći (shared read=66), ali je plan i dalje bio efikasniji od sekvencijalnog skena za ovu selektivnost.

Za upit `value < 800`, koji vraća oko 80% tabele, PostgreSQL optimizer je procenio da bi indeksni pristup generisao preveliki broj nasumičnih heap pristupa i prešao na sekvencijalno skeniranje (Seq Scan). U ovom slučaju linearno čitanje cele tabele je postalo povoljnije. Interesantno je da Seq Scan u ovom slučaju ima manji broj disk čitanja (`shared read = 0`) jer koristi podatke koji su već delimično keširani iz prethodnih testova.

5.3 Uticaj veličine tabele u odnosu na `shared_buffers`

Na manjoj tabeli (67 MB), koja u potpunosti staje u `shared_buffers` (128 MB), disk I/O je bio gotovo nevidljiv čak i pri prvom izvršavanju upita.

Nasuprot tome, na većoj tabeli (287 MB), koja značajno premašuje veličinu buffer pool-a, postaje jasno vidljiv pravi uticaj keširanja:

- Cold cache stanje generiše veliki broj disk čitanja (čak 25.792 `shared read`).
- Warm cache stanje omogućava izvršavanje istog upita bez ijednog disk pristupa.

Ovo jasno pokazuje da kada tabela ne može da stane u `shared_buffers`, PostgreSQL mnogo više zavisi od stanja keša, a performanse indeksnog pristupa postaju izrazito osetljive na to da li su relevantni blokovi već učitani u memoriju.

6. Zaključak

Cilj ovog rada bio je istraživanje uticaja PostgreSQL buffer pool-a (`shared_buffers`) i caching mehanizama na performanse upita pri radu sa tabelama različitih veličina.

Eksperimentalna analiza je sprovedena na dve tabele: manjoj (67 MB), koja u potpunosti staje u `shared_buffers` (128 MB), i većoj (287 MB), koja značajno premašuje veličinu buffer pool-a. Uporednim testiranjem cold i warm cache stanja, dobijeni su sledeći ključni rezultati:

- Na manjoj tabeli (koja staje u buffer pool) disk I/O je bio minimalan čak i pri prvom izvršavanju, te razlika između cold i warm cache stanja nije bila značajna.
- Na većoj tabeli uticaj keširanja postaje veoma izražen: cold cache stanje je izazvalo veliki broj disk operacija (25.792 shared read), što je rezultiralo vremenom izvršavanja od 203,542 ms kod sekvencijalnog skena.
- Nakon učitavanja podataka u buffer pool (warm cache), isti upit je izvršen za samo 3,157 ms — preko 60 puta brže, uz potpuno eliminisanje disk I/O operacija.

Ovi rezultati jasno demonstriraju da veličina tabele u odnosu na `shared_buffers` predstavlja presudan faktor performansi. Kada tabela ne može da stane u buffer pool, sistem postaje veoma osetljiv na stanje keša — cold cache dovodi do intenzivnog disk I/O, dok warm cache omogućava gotovo isključivo memorijski pristup podacima.

Glavni zaključak rada je da buffer pool i operativni sistemski cache predstavljaju jedan od najvažnijih mehanizama za optimizaciju performansi PostgreSQL baza podataka. Njihova efikasnost zavisi od:

- odnosa veličine podataka i dodeljene memorije (`shared_buffers`),
- selektivnosti upita i izbora plana izvršavanja,
- ponovljivosti upitnih obrazaca (`workload-a`).

Rad potvrđuje da adekvatno podešavanje i praćenje stanja buffer pool-a ima veći uticaj na performanse nego što se često pretpostavlja, posebno kod tabela koje prevazilaze raspoloživu memoriju.

7. Literatura

- [1] Raghu Ramakrishnan i Johannes Gehrke. *Database Management Systems*. 3. izdanje. McGraw-Hill, 2003.
- [2] The PostgreSQL Global Development Group. *PostgreSQL 18 Documentation*. Dostupno na: <https://www.postgresql.org/docs/>.
- [3] Hironobu Suzuki. *The Internals of PostgreSQL*. Dostupno na: <https://www.interdb.jp/pg/>. 2022.
- [4] Egor Rogov. *PostgreSQL 14 Internals*. Postgres Professional, 2023.
- [5] Hans-Jürgen Schönig. *Mastering PostgreSQL 17*. 6. izdanje. Packt Publishing, 2024.